Step 0a — Quale approccio: decision model, classificatore stile FactCG/MiniCheck, o LLM con testa nuova?

Obiettivo

Decidere **da quale modello partire e come fargli produrre la risposta** quando il compito è solo binario (documento + claim → vero/falso). Questo file confronta le opzioni con i numeri disponibili e dà una raccomandazione motivata.

1. Che cosa serve davvero

Il compito si scrive in una riga:

`f(documento, claim) → P(supportato) ∈ [0, 1]` poi $\hat{y} = 1[P > \text{soglia}]$

Servono quattro cose, in quest'ordine di importanza:

Requisito	Perché	Da cosa dipende soprattutto
Ordinamento (AUC alta)	separa i claim veri dai falsi indipendentemente dalla soglia	capacità del backbone + qualità dei dati
Calibrazione	una soglia scelta su un dominio deve valere sugli altri	loss, dati bilanciati, inizializzazione
Documenti lunghi	in uso il documento può essere molto più lungo del claim	contesto del backbone, chunking
Costo	numero di claim da verificare	taglia del modello, numero di forward

Non serve: rispondere ad altre domande, spiegare il verdetto, generare testo.

2. Cosa sono davvero FactCG e MiniCheck

Dalle analisi in ../models/minicheck.md e ../models/FactCG.md:

Componente	MiniCheck	FactCG	Contributo nuovo?
Backbone	RoBERTa-L (da AlignScore), DeBERTa-v3-L, Flan-T5-L	**gli stessi**	no
Testa	2 classi su `[CLS]`; per T5 i logit di due token	idem	no
Training	full fine-tuning, CE, 2 stadi	idem	no
Inferenza	chunk da ~400–550 token, `max` sui chunk, soglia 0,5	idem (soglia anche tarata)	no
Dati	**C2D + D2C** (14k sintetici) + 21k ANLI difficili	**CG2C-MHQA + CG2C-Doc** + dati MiniCheck	**sì: è tutto il contributo**

Quindi "creare qualcosa come FactCG o MiniCheck" significa in pratica **due scelte indipendenti**: 1. la **ricetta di dati** (sintetici con negativi costruiti togliendo un pezzo di evidenza); 2. il **backbone** su cui applicarla.

La prova più chiara viene da MiniCheck stesso: la stessa ricetta su un LLM da 7B (InternLM2.5-7B-chat, 63,6 zero-shot) dà **Bespoke-MiniCheck-7B, 77,4**, contro 75,0 del suo Flan-T5 da 0,8B. A parità di dati, il backbone più grande vince di ~2,4 punti.

3. Le opzioni

ID	Backbone	Come si legge la risposta	Punto di partenza (AggreFact)	Costo inferenza	Commento
E1	DeBERTa-v3-large (0,4B), stile FactCG-DBT	testa nuova `pooler + Linear(1024→2)`	75,6 dopo il training di FactCG	bassissimo (anche CPU)	tetto noto ~75–76; contesto 2.048 token, chunking obbligatorio
E2	Flan-T5-large (0,8B), stile MiniCheck-FT5	logit dei token "Yes"/"No"	75,0 dopo il training	basso	stesso tetto; peggiore di E1 in FactCG
L1	LLM base (Qwen3.5-9B / Gemma-4-12B) + **testa nuova** casuale	`Linear(hidden→2)` sull'ultimo token	~50 (testa casuale)	1 forward	stessa capacità di L2, inizializzazione peggiore
L2	LLM base + **logit delle lettere** (come Bespoke)	logit di "A"/"B" o "Yes"/"No"	~70–73 tipico per LLM 8–12B zero-shot (da misurare: B4)	1 forward	la ricetta di Bespoke-MiniCheck-7B
D3	Decision model, regola a 3 domande	3 domande, `min(p1, 1−p2, 1−p3)`	Winnow 77,42 / JPT 76,71	3 forward	il piano di ../fine_tuning/
D1	**Decision model, una sola domanda `noul`**	logit delle lettere A/B	**Winnow 76,69 / JPT 76,20**	1 forward	**raccomandato**
S	Encoder (E1) **distillato** da D1 fine-tunato	testa binaria	—	bassissimo	solo se il costo è il vincolo principale

(Punti di partenza: dev-tuned sul dev di AggreFact, vedi 00_overview.md §2.)

4. "Cambiare la testa" non cambia la capacità: le lettere sono già una testa binaria

Nel decision model, all'ultima posizione del prompt il modello produce uno stato nascosto `h` (dimensione `d`, es. 3.840) e la testa del vocabolario calcola un logit per token: `z_k = w_k · h`. Si leggono solo le due lettere:

```
P(vero) = softmax([z_falso, z_vero] / T)[vero] = σ( (w_vero − w_falso) · h / T )
```

È **esattamente** una regressione logistica su `h` con vettore di pesi `u = (w_vero − w_falso) / T` e bias 0. Cioè una testa `Linear(d → 1)`, già inizializzata.

Confronto con l'opzione L1 (testa nuova):

	Lettere (D1 / L2)	Testa nuova (L1, E1)
Funzione	`σ(u · h)`	`σ(u' · h + b)`
Parametri della testa	`d` (impliciti nei pesi del vocabolario)	`d + 1` (o `2d + 2` con 2 classi)
Inizializzazione di `u`	quella che il modello usa già per rispondere "true/false"	casuale
P(supportato) al passo 0	76–77 di BAcc (decision model)	~50 (caso)
Interazione con LoRA	LoRA sposta `h`; `u` resta quello appreso	serve addestrare anche la testa (`modules_to_save`)

	Lettere (D1 / L2)	Testa nuova (L1, E1)
Uso in inferenza	vLLM / llama.cpp come sempre (1 token, logprob)	servono classi `ForSequenceClassification` supportate per l'architettura

Esempio numerico. Winnow, una coppia supportata, `h` tale che `z_falso = 2,1`, `z_vero = 0,3` → $P(\text{vero}) = \sigma(-1{,}8) = 0{,}14$. Con la testa estratta `u = w_vero − w_falso`: `u · h = 0,3 − 2,1 = −1,8` → $\sigma(-1{,}8) = 0{,}14$. Stesso numero, stessa funzione. Una testa nuova casuale partirebbe invece da `u' · h ≈ 0 ± rumore` → $P \approx 0{,}5$ per qualsiasi coppia: si perde tutto il "sapere" del modello su che cosa significa "il documento supporta il claim" a livello di uscita, e lo si deve ricostruire con i dati.

Conclusione. Togliere la testa del vocabolario e metterne una nuova non aumenta la capacità e peggiora il punto di partenza. Le due cose utili che una "testa binaria" dà si ottengono senza cambiare nulla nel training: - **memoria in training**: si calcolano solo i 2 logit in `answer_pos`, non i ~260.000 del vocabolario su tutta la sequenza (05_training.md); - **servizio come classificatore**: alla fine si estrae `u` e si salva come `score` di un modello `ForSequenceClassification`, se il server lo supporta (08_merge_export.md).

Unico caso in cui una testa nuova ha senso: un backbone **senza** risposta naturale sì/no (encoder come DeBERTa). È il caso E1.

5. Decision model o LLM base?

Argomenti a favore del decision model (D1 rispetto a L2):

Argomento	Evidenza
Parte più in alto	76,2–76,7 con una domanda contro ~70–73 tipici di LLM generici di taglia simile (Llama-3.1-8B 70,3; Qwen2.5-7B 72,8 in leaderboard)
Probabilità già "da classificatore"	è addestrato a mettere tutta la massa su due lettere, con loss di Brier (JPT): niente massa dispersa su altri token
Serve meno dati e meno passi per arrivare allo stesso punto	da verificare con il controllo C1
Prompt già robusto a stati e campi diversi	utile se in futuro cambia il formato dell'input

Argomenti contro, da prendere sul serio:

Argomento	Evidenza
Con dati buoni la differenza di partenza può sparire	InternLM2.5-7B passa da 63,6 a 77,4 con 35k esempi: +13,8 punti. L'inizializzazione conta meno dei dati
Licenze	JPT è CC BY-NC 4.0; Winnow è Apache 2.0 ma i derivati dipendono dai dati
Formati particolari	Winnow: tokenizer convertito da GGUF, A = falso; JPT: A = sì, T = 1,087
Bias zero-shot verso "sì"	soglia ottimale 0,90–0,95 per `support_simple` (si corregge con il training)

Il modo onesto di decidere è il **controllo C1** (07_esperimenti.md): stessa ricetta, stessi iperparametri, su Gemma-4-12B base e Qwen3.5-9B base. Se il base arriva entro ~0,5 punti, il decision model non porta valore e si può scegliere il backbone per licenza e costo.

6. Decision model o encoder stile FactCG?

	Decision model 9–12B (D1)	Encoder 0,4B (E1)
AggreFact atteso dopo il training	≥ 77,4 (Bespoke è il riferimento per un LLM da 7B con la stessa ricetta)	~75–76 (FactCG 75,6 con dati simili)
Contesto	32k+ token nativi; in training 4–8k	2.048 token (DeBERTa); chunking obbligatorio
Ragionamento multi-hop fra parti lontane del documento	possibile in un solo chunk lungo	solo dentro un chunk; il `max` sui chunk non collega chunk diversi
Costo per claim	1 forward di un 12B su ~1–4k token (GPU)	1 forward di 0,4B per chunk (anche CPU)
Training	LoRA, ore su L40S	full fine-tuning, 1–4 h sul portatile
Rischio	sovra-confidenza, formato	tetto basso

L'encoder vince solo sul costo. Se il costo conta, la strada migliore è **S** (**distillazione**): il decision model fine-tunato etichetta con probabilità morbide molti più esempi di quanti si possano generare con un LLM giudice, e l'encoder impara da quelle. È lo stesso principio per cui MiniCheck usa dati sintetici: un insegnante più forte produce i dati per uno studente piccolo.

7. Raccomandazione

```
Compito binario, documento fornito
│
├── Serve la massima accuratezza e c'è una GPU in produzione?
│      └── SÌ → D1: decision model + 1 domanda noul + loss sulle lettere + dati MiniCheck/FactCG
│               (JPT-9B per iterare, Winnow-12B come modello finale)
│
├── Serve verificare milioni di claim a costo minimo, anche su CPU?
│      └── D1 come insegnante → S: encoder (DeBERTa-v3-large o ModernBERT-large) distillato
│
└── Mai: LLM + testa nuova casuale (L1). Stessa capacità di D1/L2, partenza peggiore.
```

In concreto: 1. **Partire dal decision model** (non da un LLM a cui cambiare la testa) e **concentrarsi su una sola domanda `noul`**. È la risposta alla domanda iniziale. 2. **Prendere da FactCG/MiniCheck tutto ciò che non è il backbone**: dati C2D/D2C/CG2C, ANLI difficile, curriculum a due stadi, chunking con `max`, obiettivo di soglia vicina a 0,5. 3. **Misurare, non assumere**: la fase A di 07_esperimenti.md addestra con gli stessi dati D1, L2 (modello base) ed E1 (encoder). Costa ~3 corse in più e chiude la questione con numeri propri.

8. Cosa si perde specializzando

- Il modello fine-tunato **non è più un decision model generale** affidabile: altre domande possono peggiorare (non lo si misura e non lo si garantisce). Va dichiarato nel nome e nella model card (es. `winnow-12b-fc`, non `winnow-12b-v2`).
- La regola a 3 domande (contraddetto vs non stabilito) non è più disponibile: se in futuro serve distinguere "falso" da "non verificabile", serve il piano di ../fine_tuning/ o una seconda domanda addestrata apposta.

Criteri per passare allo step successivo

- Scelta del backbone principale (D1 su JPT per il pilota, Winnow per la versione finale) e dei controlli (L2, E1) registrata.
- Deciso se la distillazione (S) è un requisito o un'opzione.